CSS Advanced Selectors

Attribute Selectors Guide

Master selecting elements based on their HTML attributes

Easy English • Practical Examples • Ready to Use

Contents

Chapter 1

Introduction to Attribute Selectors

1.1 What are Attribute Selectors?

Attribute selectors are CSS selectors that target HTML elements based on their attributes and attribute values. Instead of selecting elements by tag name, class, or ID, attribute selectors allow you to select elements based on specific attributes like `href`, `type`, `data-*`, and many more.

Think of it as: "Select all elements that have a specific attribute or attribute value."

Key Concept

Attribute selectors help you target elements based on the attributes they contain, such as `href`, `type`, `placeholder`, `data-*`, etc., without needing to add classes or IDs to your HTML.

1.2 Why Use Attribute Selectors?

- Target elements without adding extra classes to HTML
- Select form inputs based on their type
- Style links based on their destination
- Apply styles based on data attributes
- Make your CSS more flexible and reusable
- Reduce HTML bloat with unnecessary classes

1.3 Common Attributes

- `href` - URL in links
- `type` - Input type (text, password, email, checkbox, etc.)
- `placeholder` - Placeholder text in inputs
- `disabled` - Disabled form elements

- `data-*` - Custom data attributes
- `src` - Image source
- `alt` - Alternative text
- `title` - Tooltip text

Chapter 2

Types of Attribute Selectors

2.1 1. Presence Selector [attribute]

2.1.1 Syntax

```
element[attribute] { }
```

2.1.2 Explanation

This selector targets any element that HAS a specific attribute, regardless of the attribute's value. It simply checks if the attribute exists.

In simple words: "Select elements that have this attribute"

2.1.3 Example

CSS Code

```
input[disabled] {
    opacity: 0.5;
    cursor: not-allowed;
}
```

HTML:

```
<input type="text" disabled>
<input type="text">
```

Result: Only the first input with the `disabled` attribute becomes faded (opacity 0.5). The second input remains normal.

Practical Use Case

Use this to style disabled form inputs, checkboxes with the `checked` attribute, links with the `title` attribute, images with the `alt` attribute, etc.

2.2 2. Exact Value Selector [attribute=”value”]

2.2.1 Syntax

```
element[attribute="value"] { }
```

2.2.2 Explanation

This selector targets elements whose attribute matches EXACTLY the specified value. The entire value must match perfectly.

In simple words: ”Select elements where the attribute equals exactly this value”

2.2.3 Example

CSS Code

```
input[type="email"] {
    border: 2px solid blue;
}
```

HTML:

```
<input type="email">
<input type="text">
<input type="email">
```

Result: Only the email inputs get a blue border. The text input is unaffected.

Another example with links:

CSS Code

```
a[target="_blank"] {
    color: red;
    text-decoration: underline;
}
```

HTML:

```
<a href="page.html">Internal Link</a>
<a href="page.html" target="_blank">External Link</a>
```

Result: Only the link with `target="_blank"` becomes red with underline.

2.3 3. Substring Match: Starts With [attribute^="value"]

2.3.1 Syntax

```
element[attribute^="value"] { }
```

2.3.2 Explanation

The caret symbol (^) selects elements whose attribute value STARTS WITH the specified text. It doesn't need to be the entire value, just the beginning.

In simple words: "Select elements where the attribute starts with this text"

2.3.3 Example

CSS Code

```
a[href^="https"] {
    color: green;
}
```

HTML:

```
<a href="https://google.com">Secure Link</a>
<a href="http://google.com">Non-secure Link</a>
<a href="https://github.com">GitHub</a>
```

Result: Both links starting with "https" become green. The "http" link remains the default color.

Another example with file types:

CSS Code

```
a[href^="mailto:"] {
    color: purple;
}
```

HTML:

```
<a href="mailto:info@example.com">Email Us</a>
<a href="tel:123-456-7890">Call Us</a>
```

Result: Only the email link becomes purple.

2.4 4. Substring Match: Ends With [attribute$=”value”]

2.4.1 Syntax

```
element[attribute$="value"] { }
```

2.4.2 Explanation

The dollar sign ($) selects elements whose attribute value ENDS WITH the specified text. Perfect for matching file extensions or domain names.

In simple words: "Select elements where the attribute ends with this text"

2.4.3 Example

CSS Code

```
a[href$=".pdf"] {
    color: red;
}
```

HTML:

```
<a href="document.pdf">Download PDF</a>
<a href="image.jpg">View Image</a>
<a href="file.pdf">Another PDF</a>
```

Result: Both links ending with ".pdf" become red. The image link remains normal.

Another example with file downloads:

CSS Code

```
a[href$=".zip"] {
    font-weight: bold;
}

a[href$=".exe"] {
    color: red;
    background: yellow;
}
```

2.5 5. Substring Match: Contains [attribute*=”value”]

2.5.1 Syntax

```
element[attribute*="value"] { }
```

2.5.2 Explanation

The asterisk (*) selects elements whose attribute value CONTAINS the specified text anywhere in the value. It doesn't need to be at the start or end.

In simple words: "Select elements where the attribute contains this text anywhere"

2.5.3 Example

CSS Code

```
a[href*="github"] {
    color: #333;
    border: 1px solid #333;
    padding: 5px;
}
```

HTML:

```
<a href="https://github.com/user">GitHub Profile</a>
<a href="https://github.com/user/project">Project</a>
<a href="https://google.com">Google</a>
```

Result: Both links containing "github" in the href get the special styling.

Another example with classes:

CSS Code

```
div[class*="box"] {
    border: 1px solid #ccc;
}
```

HTML:

```
<div class="box">Simple box</div>
<div class="alert-box">Alert box</div>
<div class="message">Message</div>
```

Result: Both divs with "box" anywhere in their class name get a border.

2.6 6. Word Match [attribute~=”value”]

2.6.1 Syntax

```
element[attribute~="value"] { }
```

2.6.2 Explanation

The tilde (~) selects elements whose attribute contains a specific word as a complete word (separated by spaces). This is useful for attributes with multiple space-separated values like class names.

In simple words: "Select elements where the attribute contains this exact word (space-separ

2.6.3 Example

CSS Code

```
a[rel~="external"] {
    color: blue;
    text-decoration: underline;
}
```

HTML:

```
<a href="page.html" rel="external">External Link</a>
<a href="page.html" rel="internal external">Internal External</a>
<a href="page.html" rel="internal">Internal Link</a>
```

Result: The first two links have "external" as a complete word in the rel attribute, so they get styled. The third link doesn't get the styling.

Key Difference

Contains (*): "external" matches "myexternal" and "externalize"
Word Match (~): "external" matches ONLY as a separate word

2.7 7. Hyphen Match [attribute|=”value”]

2.7.1 Syntax

```
element[attribute|="value"] { }
```

2.7.2 Explanation

The pipe (|) selects elements whose attribute value exactly matches the specified value OR starts with the value followed by a hyphen. This is useful for language codes like "en-US", "en-GB", etc.

In simple words: "Select elements where the attribute is exactly this or starts with this-"

2.7.3 Example

CSS Code

```
a[lang|="en"] {
    color: green;
}
```

HTML:

```
<a lang="en">English</a>
<a lang="en-US">American English</a>
<a lang="en-GB">British English</a>
<a lang="fr">French</a>
```

Result: All links with "en" or "en-" become green. The French link stays normal.

Another example with data attributes:

CSS Code

```
div[data-lang|="es"] {
    font-style: italic;
}
```

HTML:

```
<div data-lang="es">Spanish</div>
<div data-lang="es-MX">Mexican Spanish</div>
<div data-lang="pt">Portuguese</div>
```

2.8 8. Case-Insensitive Match [attribute=”value” i]

2.8.1 Syntax

```
element[attribute="value" i] { }
```

2.8.2 Explanation

The lowercase i flag makes the attribute value matching case-insensitive. So "test", "Test", and "TEST" all match.

In simple words: "Select elements regardless of uppercase or lowercase in the attribute value"

2.8.3 Example

CSS Code

```
input[type="email" i] {
    border: 2px solid blue;
}
```

HTML:

```
<input type="email">
<input type="Email">
<input type="EMAIL">
```

Result: All three inputs get a blue border, even though the type values have different cases.

Another example with file extensions:

CSS Code

```
a[href$=".pdf" i] {
    color: red;
}
```

HTML:

```
<a href="document.pdf">PDF</a>
<a href="file.PDF">PDF</a>
<a href="guide.Pdf">PDF</a>
```

Result: All three links become red, regardless of the PDF extension case.

Chapter 3

Practical Examples and Use Cases

3.1 Example 1: Style Form Inputs

HTML Structure

```
<form>
    <input type="text" placeholder="Name">
    <input type="email" placeholder="Email">
    <input type="password" placeholder="Password">
    <input type="checkbox"> Remember me
    <input type="submit" value="Login">
    <input type="reset" value="Clear">
</form>
```

Style different input types with attribute selectors:

CSS Attribute Selectors

CSS with Attribute Selectors

```
/* Style all text inputs */
input[type="text"],
input[type="email"],
input[type="password"] {
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 4px;
    font-size: 14px;
}

/* Style email input specifically */
input[type="email"] {
    background-color: #fff9e6;
}

/* Style submit button */
input[type="submit"] {
    background-color: blue;
    color: white;
    padding: 10px 20px;
    border: none;
    cursor: pointer;
}

/* Style reset button */
input[type="reset"] {
    background-color: gray;
    color: white;
    padding: 10px 20px;
    border: none;
    cursor: pointer;
}
```

3.2 Example 2: Style Links by Protocol

HTML Structure

```
<nav>
    <a href="https://google.com">Google</a>
    <a href="mailto:info@example.com">Email</a>
    <a href="tel:123-456-7890">Call</a>
    <a href="/internal">Internal Link</a>
</nav>
```

Style different link types:

CSS with Attribute Selectors

```
/* External HTTPS links */
a[href^="https://"] {
    color: green;
}

/* Email links */
a[href^="mailto:"] {
    color: blue;
}

/* Phone links */
a[href^="tel:"] {
    color: red;
}

/* Internal links */
a[href^="/"] {
    color: purple;
}

/* PDF downloads */
a[href$=".pdf"] {
    font-weight: bold;
    border-bottom: 2px dotted orange;
}
```

3.3 Example 3: Data Attribute Styling

HTML Structure

```
<div class="status" data-status="success">Success!</div>
<div class="status" data-status="error">Error!</div>
<div class="status" data-status="warning">Warning!</div>
<div class="status" data-status="info">Info</div>
```

Style based on custom data attributes:

CSS with Attribute Selectors

```
/* Success status */
[data-status="success"] {
    background-color: #d4edda;
    color: #155724;
    border: 1px solid #c3e6cb;
    padding: 12px;
    border-radius: 4px;
}

/* Error status */
[data-status="error"] {
    background-color: #f8d7da;
    color: #721c24;
    border: 1px solid #f5c6cb;
    padding: 12px;
    border-radius: 4px;
}

/* Warning status */
[data-status="warning"] {
    background-color: #fff3cd;
    color: #856404;
    border: 1px solid #ffeeba;
    padding: 12px;
    border-radius: 4px;
}

/* Info status */
[data-status="info"] {
    background-color: #d1ecf1;
    color: #0c5460;
    border: 1px solid #bee5eb;
    padding: 12px;
    border-radius: 4px;
}
```

3.4 Example 4: Accessibility with Disabled Elements

HTML Structure

```
<form>
    <input type="text" placeholder="Active">
    <input type="text" placeholder="Disabled" disabled>
    <button>Active</button>
    <button disabled>Disabled</button>
</form>
```

Make disabled elements visually distinct:

CSS with Attribute Selectors

```
/* Style disabled inputs */
input[disabled],
button[disabled] {
    opacity: 0.6;
    background-color: #f0f0f0;
    cursor: not-allowed;
    color: #999;
}

/* Add hover effect to active inputs */
input:not([disabled]):hover {
    border-color: blue;
}

/* Add hover effect to active buttons */
button:not([disabled]):hover {
    background-color: blue;
    color: white;
}
```

3.5 Example 5: Image Gallery with Alt Text

HTML Structure

```
<img src="photo1.jpg">
<img src="photo2.jpg" alt="Nature">
<img src="photo3.jpg" alt="City">
```

Highlight images with alt text:

CSS with Attribute Selectors

```
/* All images */
img {
    border: 1px solid #ccc;
}

/* Images WITH alt text (accessible) */
img[alt] {
    border: 2px solid green;
    opacity: 1;
}

/* Images WITHOUT alt text (accessibility issue) */
img:not([alt]) {
    border: 2px solid red;
    opacity: 0.5;
}
```

Chapter 4

Quick Reference

4.1 Attribute Selector Comparison

Type	Syntax	What It Does	Example
Presence	[attr]	Has the attribute	[disabled]
Exact	[attr="val"]	Attribute equals	[type="email"]
Starts	[attr^="val"]	Attribute starts	[href^="https"]
Ends	[attr$="val"]	Attribute ends	[href$=".pdf"]
Contains	[attr*="val"]	Attribute contains	[class*="box"]
Word	[attr~="val"]	Exact word	[rel~="external"]
Hyphen	[attr|="val"]	Exact or hyphen	[lang|="en"]
Case Insensitive	[attr="val" i]	Ignores case	[type="email" i]

4.2 When to Use Each Selector

- **Presence [attr]:** Check if attribute exists, like [checked], [disabled], [required]
- **Exact [attr="val"]:** Match exact values, perfect for input types
- **Starts [^="val"]:** Match protocols like "https://", "mailto:"
- **Ends [$="val"]:** Match file extensions like ".pdf", ".jpg"
- **Contains [*="val"]:** Match partial strings anywhere in the value
- **Word [~="val"]:** Match complete words in space-separated values
- **Hyphen [|="val"]:** Match language codes and hyphenated values

- **Case Insensitive [i]:** Ignore uppercase/lowercase differences

Chapter 5

Advanced Combinations

5.1 Combining Attribute Selectors

You can combine multiple attribute selectors on a single element:

```
/* Input that is both type="text" AND has placeholder */
input[type="text"][placeholder] {
    background-color: #f9f9f9;
}

/* Link that contains github AND is external */
a[href*="github"][target="_blank"] {
    color: blue;
}
```

5.2 Attribute Selectors with Other Selectors

Combine attribute selectors with classes, IDs, and pseudo-classes:

```
/* Links in a specific class */
.nav a[href^="https://"] {
    color: green;
}

/* Required form inputs */
input[required]:focus {
    border-color: blue;
    box-shadow: 0 0 5px blue;
}

/* Disabled buttons */
button[disabled]:hover {
    cursor: not-allowed;
}

/* Images without alt (accessibility check) */
img:not([alt]) {
    border: 2px solid red;
}
```

5.3 Real-World Pattern Examples

5.3.1 Icon Links

```
/* Add icon before external links */
a[href^="https://"]::before {
    content: "";
    margin-right: 5px;
}

/* Add icon before PDF links */
a[href$=".pdf"]::before {
    content: "";
    margin-right: 5px;
}

/* Add icon before email links */
a[href^="mailto:"]::before {
    content: "";
    margin-right: 5px;
}
```

5.3.2 Form Validation Styles

```
/* Required inputs */
input[required] {
    border: 1px solid #ccc;
}

/* Required inputs with value */
input[required]:valid {
    border: 2px solid green;
    background: #f0f9ff;
}

/* Required inputs empty */
input[required]:invalid {
    border: 2px solid red;
    background: #fff0f0;
}
```

Chapter 6

Tips and Best Practices

6.1 Tip 1: Use Attribute Selectors to Reduce Classes

Instead of adding extra classes, use attribute selectors on existing HTML attributes:

```
/* Instead of: class="email-input" */
/* Use: */
input[type="email"] { }

/* Instead of: class="external-link" */
/* Use: */
a[target="_blank"] { }
```

6.2 Tip 2: Combine with Pseudo-Classes

Make your selectors more powerful by combining with pseudo-classes:

```
/* Focus state for email inputs */
input[type="email"]:focus {
    outline: none;
    border-color: blue;
}

/* Hover state for external links */
a[href^="https://"]:hover {
    text-decoration: underline;
}
```

6.3 Tip 3: Use Data Attributes for Styling

Store styling information in data attributes instead of classes:

```
/* HTML: <div data-theme="dark"> */
[data-theme="dark"] {
    background-color: #333;
    color: white;
}
```

```
[data-theme="light"] {
    background-color: #fff;
    color: #333;
}
```

6.4 Tip 4: Be Careful with Substring Matching

Substring matching can be too broad sometimes:

```
/* RISKY - Matches "testing", "mytest", etc. */
[href*="test"] { }

/* BETTER - Be more specific */
[href^="http://test."] { }
[href*="test.example.com"] { }
```

6.5 Tip 5: Use Attribute Selectors for Accessibility

Highlight elements missing accessibility attributes:

```
/* Highlight images without alt text */
img:not([alt]) {
    outline: 3px solid red;
}

/* Highlight links without title */
a:not([title]) {
    outline: 3px solid orange;
}
```

Chapter 7

Common Mistakes to Avoid

7.1 Mistake 1: Confusing Exact Match and Contains

```
/* WRONG - Only matches exactly "test" */
[href="test"] { }

/* RIGHT - Matches "test" anywhere */
[href*="test"] { }
```

7.2 Mistake 2: Forgetting Quotes

```
/* WRONG - Invalid syntax */
input[type=email] { }

/* RIGHT - Always use quotes */
input[type="email"] { }
```

7.3 Mistake 3: Confusing Word and Contains

```
/* Contains - matches "external" anywhere */
[rel*="external"] { }  /* matches "external-link" */

/* Word - matches "external" as complete word */
[rel~="external"] { }  /* matches "rel='external'" */
```

7.4 Mistake 4: Case Sensitivity Issues

```
/* WRONG - Won't match "Email" or "EMAIL" */
input[type="email"] { }

/* RIGHT - Matches regardless of case */
input[type="email" i] { }
```

7.5 Mistake 5: Using Attribute Selectors for Everything

```
/* Overuse - Too complex */
div[class~="container"][id^="main"][data-active="true"] { }

/* Better - Keep it simple */
.container#main { }
```

Chapter 8

Summary

What You Learned

8 Types of Attribute Selectors:

1. **Presence [attr]** - Element has the attribute
2. **Exact [attr="val"]** - Attribute equals value
3. **Starts [^="val"]** - Attribute starts with value
4. **Ends [$="val"]** - Attribute ends with value
5. **Contains [*="val"]** - Attribute contains value
6. **Word [~="val"]** - Attribute contains word
7. **Hyphen [|="val"]** - Attribute is value or value-
8. **Case Insensitive [i]** - Match ignoring case

Attribute selectors are incredibly useful for:

- Reducing unnecessary classes in HTML
- Styling form inputs based on type
- Creating links with icon decorations
- Working with data attributes
- Improving accessibility checking

Master attribute selectors and write cleaner, more efficient CSS!

Chapter 9

Practice Exercises

9.1 Exercise 1: Identify the Selector

For each selector, explain what it does:

1. a[href^="https"]

2. input[type="submit"]

3. div[data-toggle*="collapse"]

4. img[alt]

5. a[href$=".pdf"]

9.2 Exercise 2: Write the Selector

Write CSS selectors for the following:

1. Select all password inputs

2. Select all links that open in a new window (target="$_blank$")$Selectallelementswithadata-statusattribute$

3. Select all images missing alt text

4. Select all links pointing to PDF files

5. Select all disabled buttons

9.3 Exercise 3: Create a Style Sheet

Create a complete stylesheet for a form using attribute selectors:

- Style text, email, and password inputs differently
- Style submit and reset buttons
- Style disabled inputs
- Add focus states

Practice these exercises to become confident with attribute selectors!